

Project - High Level Design

Dockerized Marketing PHP Larvel Service

Institution Name: Medicaps University – Datagami Skill Based Course

Sr no	Student Name	Enrolment Number
1	YASH YADAV	EN22CS3011113
2	AYUSH MISHRA	EN22EL301020
3	VISHAL	EN22CS3011091
4	ASHUTOSH KUMAR	EN22EL301019
5	HARSH MANDLOI	EN22ME304035
6	AYUSH MODI	EN21CS301196

Group Name: 13D10

Project Number: DO-26

Industry Mentor Name:

University Mentor Name: PROF. AVNESH JOSHI

Academic Year: 2025-2026

Table of Contents

1. Project Overview.
 - 1.1. Problem Statement.
 - 1.2. Objectives
 - 1.3. Technology stack.
2. System Architecture & Block Diagram.
 - 2.1. High Level Architecture Block Diagram
 - 2.2. Request Flow.
3. Complete Project Tree Structure
4. Multi-Stage Dockerfile
 - 4.1. Why Multi-Stage Build
 - 4.2. Complete Dockerfile Code
5. Docker Compose Configuration
 - 5.1. Service Overview
 - 5.2. Complete Docker-Compose.yml
6. Nginx Configuration
7. Environmental Configuration
8. AWS Infrastructure
9. Complete Deployment Steps
10. Industry Best Practices Implemented
11. GitHub Repository
12. Project Summary

1. Project Overview

1.1 Problem Statement

Containerize a Marketing service built with PHP Laravel using industry best practices for Docker. The project includes writing multi-stage Dockerfiles to optimize image size and security, alongside a Docker Compose configuration to orchestrate the application and its database dependencies. This ensures a consistent, portable development and production environment across the entire team.

1.2 Objectives

- Containerize a PHP Laravel Marketing application using Docker
- Write a multi-stage Dockerfile to optimize image size and security
- Use Docker Compose to orchestrate multiple services (Nginx, PHP, MySQL, Redis)
- Deploy the application live on AWS EC2 Free Tier
- Ensure a consistent, portable environment for development and production
- Push the complete project to GitHub for version control

1.3 Technology Stack

Layer	Technology	Version	Purpose
Application	PHP Laravel	11.x	Backend Web Framework
Language	PHP	8.2	Server-side Language
Web Server	Nginx	Alpine (Latest)	Reverse Proxy & HTTP Server

Layer	Technology	Version	Purpose
Database	MySQL	8.0	Relational Database
Cache / Session	Redis	Alpine (Latest)	Cache, Sessions & Queues
Containerization	Docker	24.x	Container Runtime
Orchestration	Docker Compose	1.x	Multi-container Management
Cloud	AWS EC2	t3.micro	Cloud Virtual Machine
OS	Ubuntu	24.04 LTS	Server Operating System
Version Control	Git + GitHub	Latest	Source Code Management

2. System Architecture & Block Diagram

2.1 High Level Architecture Block Diagram

USER / BROWSER

HTTP Request → http://13.126.33.236:80

↕ Port 80 (Public Internet)

AWS SECURITY GROUP (marketing-laravel-sg)

Firewall: Ports 22 (SSH), 80 (HTTP), 8000, 3306, 6379

↕ Allowed through Firewall

AWS EC2 Instance | t3.micro | Ubuntu 24.04 LTS | Elastic IP: 13.126.33.236

Docker Network: marketing_network (bridge driver)

NGINX	PHP-FPM	MYSQL	REDIS
Web Server	Laravel App	Database	Cache/Session
nginx:alpine	php:8.2-alpine	mysql:8.0	redis:alpine
Port: 80:80	Port: 9000	Port: 3306	Port: 6379
marketing_nginx	marketing_app	marketing_db	marketing_redis

Persistent Volume: db_data → MySQL data survives container restarts

2.2 Request Flow

Step	Component	Action	Port
1	Browser / Client	User sends HTTP request to Elastic IP	80
2	AWS Security Group	Firewall validates and allows port 80 inbound traffic	80
3	Nginx Container	Receives request, acts as reverse proxy	80
4	PHP-FPM Container	Nginx forwards PHP requests via FastCGI protocol	9000
5	Laravel Application	Processes business logic, handles routing	-
6	MySQL Container	Laravel queries the database when needed	3306
7	Redis Container	Session read/write and cache operations	6379
8	Laravel Application	Prepares and returns HTTP response	-
9	Nginx Container	Sends response back to the client	80
10	Browser / Client	Renders the Laravel welcome page	-

3. Complete Project Tree Structure

The following is the complete directory and file structure of the marketing-app project:

```
marketing-app/          ← Root project directory
|
├── app/                 ← Laravel application core
|   ├── Console/         ← Artisan commands
|   ├── Exceptions/
|   |   └── Handler.php   ← Exception handler
|   ├── Http/
|   |   ├── Controllers/
|   |   |   └── Controller.php ← Base controller
|   |   ├── Middleware/   ← HTTP middleware
|   |   └── Kernel.php    ← HTTP kernel
|   ├── Models/
|   |   └── User.php       ← User model
|   └── Providers/
|       └── AppServiceProvider.php ← Service providers
|
├── bootstrap/
|   ├── app.php           ← Application bootstrapper
|   └── cache/            ← Framework bootstrap cache
|
├── config/               ← Configuration files
|   └── app.php           ← App settings
```

- | |— cache.php ← Cache config (Redis)
- | |— database.php ← DB config (MySQL)
- | |— queue.php ← Queue config (Redis)
- | |— session.php ← Session config (Redis)
- |
- |— database/
 - | |— factories/ ← Model factories for testing
 - | |— migrations/ ← Database migrations
 - | | |— 0001_01_01_000000_create_users_table.php
 - | | |— 0001_01_01_000001_create_cache_table.php
 - | | |— 0001_01_01_000002_create_jobs_table.php
 - | |— seeders/ ← Database seeders
- |
- |— docker/ ← Docker configuration directory
- | |— nginx/
 - | |— nginx.conf ← Nginx virtual host config
- |
- |— public/ ← Web root (Nginx points here)
 - | |— index.php ← Application entry point
 - | |— .htaccess ← Apache rewrite rules
- |
- |— resources/
 - | |— css/ ← Stylesheets
 - | |— js/ ← JavaScript files
 - | |— views/
 - | |— welcome.blade.php ← Default welcome page

| ├── routes/

| | ├── api.php ← API routes

| | ├── console.php ← Console routes

| | └── web.php ← Web routes

|

| ├── storage/

| | ├── app/ ← Application storage

| | ├── framework/ ← Framework cache, sessions, views

| | └── logs/

| | └── laravel.log ← Application logs

|

| ├── tests/ ← Unit and feature tests

|

| ├── vendor/ ← Composer packages (gitignored)

|

| ├── .env ← Environment variables (gitignored)

| ├── .env.example ← Environment variable template

| ├── .gitignore ← Git ignore rules

|

| ├── Dockerfile ← Multi-stage Docker build file

| ├── docker-compose.yml ← Container orchestration config

|

| ├── artisan ← Laravel CLI tool

| ├── composer.json ← PHP dependencies

| ├── composer.lock ← Dependency lock file

└─ package.json

← Node.js dependencies

4. Multi-Stage Dockerfile

4.1 Why Multi-Stage Build?

Stage	Base Image	Purpose	Includes
Stage 1 (builder)	php:8.2-fpm-alpine	Build the application	Composer, Node, npm, all dev tools, all PHP extensions
Stage 2 (production)	php:8.2-fpm-alpine	Run the application	Only runtime deps, non-root user, copied built files

- Smaller final image — build tools are not included in production
- Better security — non-root user 'www' (UID 1000) runs the app
- Faster deployments — smaller images pull and start faster
- Layer caching — composer.json copied before source for efficient rebuilds

4.2 Complete Dockerfile Code

```
# =====  
# Stage 1: BUILD STAGE  
# =====  
FROM php:8.2-fpm-alpine AS builder  
  
# Install system dependencies including oniguruma for mbstring  
RUN apk add --no-cache \  
    curl \  
    libpng-dev \  

```

```
libxml2-dev \  
oniguruma-dev \  
zip \  
unzip \  
git \  
nodejs \  
npm
```

```
# Install PHP extensions
```

```
RUN docker-php-ext-install \  

```

```
pdo \  
pdo_mysql \  
mbstring \  
exif \  
pcntl \  
bcmath \  
gd \  
xml
```

```
# Install Composer from official image
```

```
COPY --from=composer:latest /usr/bin/composer /usr/bin/composer
```

```
WORKDIR /var/www
```

```
# Copy composer files first (for Docker layer caching)
```

```
COPY composer.json composer.lock ./
```

Install PHP dependencies (no dev dependencies)

RUN composer install \

--no-interaction \

--no-plugins \

--no-scripts \

--no-dev \

--prefer-dist

Copy all project source files

COPY . .

Set correct file ownership and permissions

RUN chown -R www-data:www-data /var/www \

&& chmod -R 755 /var/www/storage

=====

Stage 2: PRODUCTION STAGE

=====

FROM php:8.2-fpm-alpine AS production

Only install runtime dependencies (no build tools)

RUN apk add --no-cache \

libpng-dev \

libxml2-dev \

oniguruma-dev

Install only required PHP extensions

```
RUN docker-php-ext-install \
```

```
pdo \
```

```
pdo_mysql \
```

```
mbstring \
```

```
bcmath \
```

```
gd \
```

```
xml
```

```
# Create non-root user for security best practice
```

```
RUN addgroup -g 1000 -S www && \
```

```
adduser -u 1000 -S www -G www
```

```
WORKDIR /var/www
```

```
# Copy ONLY built files from builder stage
```

```
COPY --from=builder --chown=www:www /var/www .
```

```
# Switch to non-root user
```

```
USER www
```

```
EXPOSE 9000
```

```
CMD ["php-fpm"]
```

5. Docker Compose Configuration

5.1 Services Overview

Service	Container Name	Image	Ports	Role
app	marketing_app	Custom Dockerfile	9000 (internal)	PHP-FPM Laravel Application
nginx	marketing_nginx	nginx:alpine	80:80 (public)	Web Server / Reverse Proxy
db	marketing_db	mysql:8.0	3306:3306	MySQL Relational Database
redis	marketing_redis	redis:alpine	6379:6379	Cache / Session / Queue Backend

5.2 Complete docker-compose.yml

```
version: '3.8'
```

```
services:
```

```
# — PHP Laravel Application —
```

```
app:
```

```
  build:
```

```
    context: .
```

```
    dockerfile: Dockerfile
```

```
    target: production
```

container_name: marketing_app

restart: unless-stopped

working_dir: /var/www

volumes:

- ./storage:/var/www/storage
- ./bootstrap/cache:/var/www/bootstrap/cache

networks:

- marketing_network

depends_on:

- db
- redis

environment:

- APP_ENV=production
- APP_DEBUG=false

— Nginx Web Server —

nginx:

image: nginx:alpine

container_name: marketing_nginx

restart: unless-stopped

ports:

- "80:80"

volumes:

- ./:/var/www
- ./docker/nginx/nginx.conf:/etc/nginx/conf.d/default.conf

networks:

- marketing_network

depends_on:

- app

— MySQL Database —

db:

image: mysql:8.0

container_name: marketing_db

restart: unless-stopped

environment:

MYSQL_DATABASE: marketing_db

MYSQL_ROOT_PASSWORD: root_password

MYSQL_USER: marketing_user

MYSQL_PASSWORD: marketing_password

volumes:

- db_data:/var/lib/mysql

ports:

- "3306:3306"

networks:

- marketing_network

— Redis Cache —

redis:

image: redis:alpine

container_name: marketing_redis

restart: unless-stopped

ports:

- "6379:6379"

networks:

- marketing_network

volumes:

db_data:

driver: local

networks:

marketing_network:

driver: bridge

6. Nginx Configuration

File path: docker/nginx/nginx.conf

```
server {  
    listen 80;  
  
    index index.php index.html;  
  
    error_log /var/log/nginx/error.log;  
  
    access_log /var/log/nginx/access.log;  
  
    root /var/www/public;  
  
  
    # Pass PHP files to PHP-FPM container on port 9000  
  
    location ~ \.php$ {  
        try_files $uri =404;  
  
        fastcgi_split_path_info ^(.+\.(php|php5|php7|php8|php9|php-[0-9]+))(/.+)$;  
  
        fastcgi_pass app:9000;  
  
        fastcgi_index index.php;  
  
        include fastcgi_params;  
  
        fastcgi_param SCRIPT_FILENAME $document_root$fastcgi_script_name;  
        fastcgi_param PATH_INFO $fastcgi_path_info;  
    }  
  
  
    # Handle Laravel's URL routing  
  
    location / {  
        try_files $uri $uri /index.php?$query_string;  
  
        gzip_static on;  
    }  
}
```

```
# Security headers

add_header X-Frame-Options "SAMEORIGIN";

add_header X-XSS-Protection "1; mode=block";

add_header X-Content-Type-Options "nosniff";


server_tokens off;      # Hide Nginx version

client_max_body_size 100M; # Max upload size

}
```

7. Environment Configuration (.env)

Key environment variables configured for Docker deployment:

Variable	Value Set	Default	Reason for Change
APP_NAME	MarketingApp	Laravel	Project identity
APP_ENV	production	local	Production mode
APP_DEBUG	false	true	Hide errors in production
APP_URL	http://13.126.33.236	http://localhost	EC2 Elastic IP
DB_CONNECTION	mysql	sqlite	Use MySQL container
DB_HOST	db	127.0.0.1	Docker service name (internal DNS)
DB_DATABASE	marketing_db	laravel	Our database name
DB_USERNAME	marketing_user	root	Dedicated DB user
SESSION_DRIVER	redis	database	Faster Redis sessions
CACHE_STORE	redis	database	Redis caching
QUEUE_CONNECTION	redis	database	Redis queue backend
REDIS_CLIENT	predis	phpredis	Pure PHP client (no extension needed)
REDIS_HOST	redis	127.0.0.1	Docker service name (internal DNS)

8. AWS Infrastructure

8.1 EC2 Instance Details

Property	Value
Instance Name	marketing-laravel-server
Instance Type	t3.micro (Free Tier Eligible)
Region	ap-south-1 (Mumbai, India)
AMI	Ubuntu Server 24.04 LTS (HVM), SSD Volume Type
Architecture	64-bit (x86)
Storage	20 GB gp3 SSD (Free tier allows up to 30 GB)
Elastic IP	13.126.33.236 (Static public IP)
Key Pair	marketing-laravel-key.pem (RSA, 4096-bit)
Connection Method	SSH via Windows PowerShell (OpenSSH)

8.2 Security Group: marketing-laravel-sg

Type	Protocol	Port	Source	Purpose
SSH	TCP	22	0.0.0.0/0	Remote access via PowerShell
HTTP	TCP	80	0.0.0.0/0	Public web traffic to Nginx
Custom TCP	TCP	8000	0.0.0.0/0	Laravel development port

Type	Protocol	Port	Source	Purpose
MySQL/Aurora	TCP	3306	0.0.0.0/0	MySQL database port
Custom TCP	TCP	6379	0.0.0.0/0	Redis cache port

9. Complete Deployment Steps

Phase	Step	Command / Action
AWS Setup	Launch EC2 Instance	AWS Console → EC2 → Launch Instance (t3.micro, Ubuntu 24.04)
AWS Setup	Create Key Pair	marketing-laravel-key.pem downloaded to C:\Users\ayush\Downloads
AWS Setup	Configure Security Group	Inbound rules: ports 22, 80, 8000, 3306, 6379
AWS Setup	Attach Elastic IP	Elastic IP 13.126.33.236 associated with instance
Connection	Fix .pem permissions	icacals + moved key to C:\Users\ayush\.ssh\
Connection	SSH into EC2	ssh -i marketing-laravel-key.pem ubuntu@13.126.33.236
Docker Setup	Install Docker	sudo apt install docker.io -y
Docker Setup	Install Docker Compose	sudo apt install docker-compose -y
Docker Setup	Add ubuntu to docker group	sudo usermod -aG docker ubuntu
Laravel Setup	Install PHP + Composer	sudo apt install php php-cli composer -y
Laravel Setup	Create Laravel project	composer create-project laravel/laravel marketing-app
Docker Config	Write Dockerfile	Multi-stage: builder + production stages

Phase	Step	Command / Action
Docker Config	Write docker-compose.yml	4 services: app, nginx, db, redis
Docker Config	Write nginx.conf	Created docker/nginx/nginx.conf
Laravel Config	Configure .env	Updated DB, Redis, App URL settings
Laravel Config	Install predis	composer require predis/predis
Launch	Stop Apache (port conflict)	sudo systemctl stop apache2 && disable apache2
Launch	Build and start containers	docker-compose up -d --build
Launch	Generate App Key	docker-compose exec app php artisan key:generate
Launch	Run DB Migrations	docker-compose exec app php artisan migrate --force
Launch	Cache for production	php artisan config:cache, route:cache, view:cache
GitHub	Configure git	git config --global user.name / user.email
GitHub	Push to repository	git init, add, commit, branch -M main, push

10. Industry Best Practices Implemented

Category	Best Practice	How Implemented
Docker	Multi-stage builds	Stage 1 builds, Stage 2 runs — production image has no build tools
Docker	Alpine base images	php:8.2-fpm-alpine and nginx:alpine reduce image size significantly
Docker	Non-root user	User 'www' (UID 1000) created and used to run PHP-FPM
Docker	Layer caching	composer.json copied before source code to cache dependency layer
Docker	Restart policies	All services use 'unless-stopped' for automatic recovery
Docker	Service dependencies	depends_on ensures startup order: db/redis → app → nginx
Security	Security headers	X-Frame-Options, X-XSS-Protection, X-Content-Type-Options in Nginx
Security	Hide server version	server_tokens off in Nginx config
Security	Non-root container	Application never runs as root inside container
Security	.env gitignored	Secrets and environment variables never committed to GitHub
Security	SSH key auth	EC2 access via RSA key pair only, no password auth
Database	Persistent volumes	db_data volume ensures data survives container restarts

Category	Best Practice	How Implemented
Database	Dedicated user	marketing_user instead of root for database connections
Networking	Bridge network	Isolated marketing_network for all containers
Networking	Internal DNS	DB_HOST=db and REDIS_HOST=redis use Docker's internal DNS
Performance	Redis sessions/cache	Faster than database-backed sessions and caching
Performance	Nginx static serving	gzip_static enabled for compressed static assets
Laravel	Production caching	config:cache, route:cache, view:cache for faster requests
AWS	Elastic IP	Static IP address that persists across instance stop/start
AWS	Free tier optimized	t3.micro + 20GB gp3 SSD within AWS free tier limits

11. GitHub Repository

Detail	Value
Repository URL	https://github.com/Ayush-js/marketing-laravel-docker
Branch	main
Visibility	Public
Commit Message	Initial commit - Dockerized Laravel Marketing App
Objects Pushed	81 objects (74.36 KiB)
Authentication	Personal Access Token (Classic, repo scope)

Key Files in Repository

- Dockerfile — Multi-stage Docker build configuration
- docker-compose.yml — Orchestrates all 4 containers
- docker/nginx/nginx.conf — Nginx virtual host config
- .env.example — Environment variable template (not the real .env)
- .gitignore — Excludes vendor/, .env, storage keys from git
- composer.json / composer.lock — PHP dependency definitions
- routes/web.php — Laravel application routes
- database/migrations/ — All 3 database migration files

12. Project Summary

This project successfully demonstrates the containerization of a PHP Laravel Marketing Service on AWS EC2 using Docker industry best practices. The application is fully live and accessible at <http://13.126.33.236>.

Metric	Value
Total Docker Containers	4 (app, nginx, db, redis)
Dockerfile Type	Multi-stage (builder + production)
Base Image	PHP 8.2 FPM Alpine (minimal, secure)
Database Tables Created	3 (users, cache, jobs)
Live Application URL	http://13.126.33.236
GitHub Repository	github.com/Ayush-js/marketing-laravel-docker
Files Committed	81 objects (74.36 KiB)
AWS Region	ap-south-1 (Mumbai, India)
Instance Type	t3.micro (Free Tier)
Storage	20 GB gp3 SSD
Project Status	LIVE & RUNNING ✓